

AI Assisted Coding

Assignment 7.5

Name: K. Deekshith

Ht.no: 2303A51414

Batch: 21

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Lab Objectives:

- To identify and correct syntax, logic, and runtime errors in Python programs using AI tools.
- To understand common programming bugs and AI-assisted debugging suggestions.
- To evaluate how AI explains, detects, and fixes different types of coding errors.
- To build confidence in using AI to perform structured debugging practices.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Use AI tools to detect and correct syntax, logic, and runtime errors.
- Interpret AI-suggested bug fixes and explanations.

- Apply systematic debugging strategies supported by AI-generated insights.

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument cause

unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):
```

```
    items.append(item)
```

```
    return items
```

```
print(add_item(1))
```

```
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.

Code:

```
Lab_7.5.py > add_item
1  '''
2  Task 1 (Mutable Default Argument – Function Bug)
3  Task: Analyze given code where a mutable default argument causes
4  unexpected behavior. Use AI to fix it.
5  # Bug: Mutable default argument
6  Expected Output: Corrected function avoids shared list bug.
7  '''
8 → def add_item(item, items=[]):
   items.append(item)
   return items
11 print(add_item(1))
12 print(add_item(2))
13
```

```
def add_item(item, items=None):
    if items is None:
        items = []
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python  
[1]  
[2]
```

Observation:

The default list `items = []` is created only once, so all function calls share the same list. Because the list is mutable, values added in earlier calls remain in later calls, causing unexpected accumulation. Using `items=None` and creating a new list inside the function ensures each call works with a fresh list and fixes the bug.

Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails.

Use AI to correct with tolerance.

Bug: Floating point precision issue

```
def check_sum():
```

```
    return (0.1 + 0.2) == 0.3
```

```
print(check_sum())
```

Expected Output: Corrected function

Code:

```

16
17 # Task 2 (Floating-Point Precision Error)
18 '''
19 Task: Analyze given code where floating-point comparison fails.
20 Use AI to correct with tolerance.
21 # Bug: Floating point precision issue
22 '''
23 def check_sum():
24     return (0.1 + 0.2) == 0.3
25 print(tolerance = 1e-10)

```

Output:

```

C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
True

```

Observation:

Floating-point numbers cannot always represent decimal values exactly, so $(0.1 + 0.2)$ is not exactly equal to 0.3 . Direct comparison using `==` may therefore return `False` even when the values should logically be equal.

Using a tolerance (epsilon) and checking whether the absolute difference is smaller than that tolerance correctly handles floating-point precision errors.

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case

```
def countdown(n):
```

```
    print(n)
```

```
    return countdown(n-1)
```

```
countdown(5)
```

Expected Output : Correct recursion with stopping condition.

Code:

```
29
30 # Task 3 (Recursion Error [ ] Missing Base Case)
31 '''
32 Task: Analyze given code where recursion runs infinitely due to
33 missing base case. Use AI to fix.
34 # Bug: No base case
35 Expected Output : Correct recursion with stopping condition.'''
36 def countdown(n):
37     if n == 0:
38         return
39     print(n)
40     return countdown(n-1)
41 countdown(5)
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
5
4
3
2
1
```

Observation:

The original recursive function lacks a base case, so the function keeps calling itself indefinitely, leading to infinite recursion or a recursion depth error.

Adding a stopping condition such as `if n == 0: return` ensures that the recursion terminates properly. After fixing, the function prints numbers from 5 down to 1 and then stops successfully.

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():  
    data = {"a": 1, "b": 2}  
    return data["c"]  
print(get_value())
```

Expected Output: Corrected with .get() or error handling.

Code:

```
44 # Task 4 (Dictionary Key Error)  
45 '''  
46 Task: Analyze given code where a missing dictionary key causes  
47 error. Use AI to fix it.  
48 # Bug: Accessing non-existing key  
49 Expected Output: Corrected with .get() or error handling.  
50 '''  
51 def get_value():  
52     data = {"a": 1, "b": 2}  
53     return data["c"]  
54     return data.get("c", "Key not found")  
55 print(get_value())
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python  
Key not found
```

Observation:

Accessing a non-existing dictionary key using `data["c"]` raises a `KeyError` because the key is not present in the dictionary. Using

`data.get("c", "Key not found")` safely returns a default value instead of causing an error. This fix ensures the program runs without crashing and handles missing keys gracefully.

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

```
def loop_example():
```

```
    i = 0
```

```
    while i < 5:
```

```
        print(i)
```

Expected Output: Corrected loop increments i.

Code:

```
57     '''
58     Task 5 (Infinite Loop ❌ Wrong Condition)
59     Task: Analyze given code where loop never ends. Use AI to detect
60     and fix it.
61     # Bug: Infinite loop
62     Expected Output: Corrected loop increments i.
63     '''
64     def loop_example():
65         i = 0
66         while i < 5:
67             print(i)
68             i += 1
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
0
1
2
3
4
```

Observation:

The original loop does not update the value of *i*, so the condition *i* < 5 always remains true, causing an infinite loop. Adding the increment statement *i* += 1 inside the loop ensures that *i* increases after each iteration. With this fix, the loop prints values from 0 to 4 and then terminates correctly.

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

Code:

```
71 # Task 6 (Unpacking Error | Wrong Variables)
72 '''
73 Task: Analyze given code where tuple unpacking fails. Use AI to
74 fix it.
75 # Bug: Wrong unpacking
76 Expected Output: Correct unpacking or using _ for extra values.
77 '''
78 →| a, b = (1, 2, 3)
    | a, b, _ = (1, 2, 3)
```


Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python  
1 2
```

Observation:

The tuple (1, 2, 3) contains three values, but only two variables (a, b) are provided, causing a "too many values to unpack" error.

Tuple unpacking requires the number of variables to match the number of values unless a placeholder like _ is used. Fixing it as a, b, _ = (1, 2, 3) or using three variables correctly resolves the unpacking error.

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

```
def func():
```

```
    x = 5
```

```
    y = 10
```

```
    return x+y
```

Expected Output : Consistent indentation applied.

Code:

```
80 # Task 7 (Mixed Indentation) Tabs vs Spaces)
81 '''
82 Task: Analyze given code where mixed indentation breaks
83 execution. Use AI to fix it.
84 # Bug: Mixed indentation
85
86 Expected Output : Consistent indentation applied
87 '''
```

Modify selected code

Add Context...

✓ ✕

Auto ▾

```
88 def func():
89     x = 5
90     y = 10
91     return x+y
```

Keep

Undo

📄

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
15
```

Observation:

The original code uses mixed indentation (tabs and spaces), which causes an indentation error and prevents the program from executing correctly. Python requires consistent indentation within a block to define the function body properly. Using the same indentation style (for example, four spaces for each line) fixes the issue and produces the correct output 15.

Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

```
import maths
```

```
print(maths.sqrt(16))
```

Expected Output: Corrected to import math

Code:

```
93 # Task 8 (Import Error: Wrong Module Usage)
94 ...
95 Task: Analyze given code with incorrect import. Use AI to fix.
96 # Bug: Wrong import
97 Expected Output: Corrected to import math
98 ...
```

Modify selected code

🔗 Add Context...

✓ ✕

Auto ▾

```
import maths
print(maths.sqrt(16))
99 import math
100 print(math.sqrt(16))
```

Keep

Undo

📄

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
4.0
```

Observation:

The original code fails because it uses mixed indentation, which Python does not allow within the same block. This causes an indentation error and stops the program from running correctly. Applying consistent indentation (such as four spaces for all lines inside the function) fixes the issue and allows the function to execute properly.